REST vs gRPC: An Empirical Measurement of Network Overhead in Space and Time

Kartikeya Dimri
Integrated M.Tech CSE
IIIT Bangalore
Bengaluru, India
kartikeya.dimri@iiitb.ac.in

***Abstract*—Efficient inter-service communication is a critical factor in the performance of modern distributed systems. While prior work comparing REST and gRPC primarily reports aggregate metrics such as latency and throughput, it often treats communication overhead as a black box. This work presents a systematic decomposition of communication overhead into distinct space and time components, enabling a fine-grained understanding of protocol behavior. Specifically, space overhead is separated into encoding overhead (JSON vs Protobuf) and framing overhead (HTTP/1.1 vs HTTP/2), while time overhead captures serialization and deserialization cost.**

These findings provide a structured understanding of protocol overhead by explicitly separating its components and revealing the strong influence of payload structure on both space and time costs. The results show that while framing overhead is fixed, encoding and serialization overheads vary significantly with data shape, with nested and wide structures incurring substantially higher costs. The study therefore highlights that protocol efficiency is not solely determined by the choice of REST or gRPC, but is equally dependent on how data is structured, offering practical guidance for both protocol selection and payload design in distributed systems.

***Index Terms*—REST, gRPC, distributed systems, Protocol Buffers, JSON, serialization, encoding, framing, network overhead**

I. Introduction

The rapid adoption of distributed and cloud-native architectures, particularly microservices, has fundamentally transformed modern software systems. By decomposing monolithic applications into independently deployable services, these architectures enable improved scalability, fault isolation, and development agility [1]. However, this shift also introduces a critical dependency on inter-service communication, where system performance increasingly depends on the efficiency of the underlying communication protocols [2], [3]. As microservices exchange data over the network, even small inefficiencies in communication can accumulate, significantly impacting overall system latency and resource utilization [4].

Among the available communication paradigms, REST (Representational State Transfer) and gRPC (Google Remote Procedure Call) have emerged as two dominant approaches. REST, built on HTTP/1.1 and typically using JSON for data exchange, is widely adopted due to its simplicity, flexibility, and extensive tooling support. In contrast, gRPC leverages HTTP/2 and Protocol Buffers (Protobuf), enabling compact binary serialization, multiplexed communication, and efficient streaming capabilities. These fundamental differences lead to distinct trade-offs in performance, complexity, and applicability across different system requirements [5].

TABLE I
HIGH-LEVEL COMPARISON OF REST AND GRPC

Feature	REST	gRPC
Transport protocol	HTTP/1.1	HTTP/2
Data format	JSON (text-based)	Protobuf (binary)
Serialization cost	Higher	Lower
Payload size	Larger	Smaller
Streaming support	Limited	Native

A high-level comparison of the two approaches is summarized in Table I.

Extensive prior research has compared REST and gRPC in the context of microservices [6]. Most studies focus on end-to-end performance metrics such as latency, throughput, CPU usage, and memory consumption [7], [8]. These works consistently report that gRPC outperforms REST in performance-critical scenarios, primarily due to its efficient binary serialization and HTTP/2 transport features [9], [10]. Other studies highlight the impact of communication protocols on system-level characteristics such as scalability and energy consumption, demonstrating that protocol choice can significantly influence overall system behavior [11].

Despite these contributions, existing work largely treats communication performance as a black-box phenomenon, reporting aggregate metrics without isolating the underlying causes of overhead. While some studies measure network-related factors such as payload size or response time [12], they do not explicitly decompose communication cost into its constituent components, such as encoding, transport framing, and serialization overhead. As a result, there remains limited understanding of where and why performance differences between protocols arise.

This study addresses this gap by introducing a systematic decomposition of communication overhead across both space

Code and files are available on GitHub: kartikeya-dimri/Network-Overhead-REST-vs-gRPC.

TABLE II
EXPERIMENTAL SYSTEM SPECIFICATIONS

Component	Specification
OS	Ubuntu 24.04.4 LTS
Kernel	Linux 6.8.0-85-generic
CPU	12th Gen Intel Core i7-12700 (12 cores, 24 threads, up to 4.9 GHz)
RAM	16 GB DDR
NIC	Realtek RTL8111/8168/8411 PCI Express Gigabit Ethernet Controller
NIC driver	r8169
Link	1000 Mb/s, full duplex, twisted-pair Ethernet

and time dimensions. Specifically, we separate:

- **space overhead** into:
  - encoding overhead (JSON vs. Protobuf), and
  - framing overhead (HTTP/1.1 vs. HTTP/2);
- **time overhead** into:
  - serialization and deserialization costs

All measurements are conducted in a controlled and isolated experimental environment, using minimal echo-based services to remove any external effects. Furthermore, the study evaluates overhead across varying payload sizes and structures, enabling a fine-grained analysis of protocol behavior.

This work aims to provides deeper insight into the fundamental trade-offs between REST and gRPC, contributing to a more principled understanding of inter-service communication efficiency in distributed systems.

II. METHODOLOGY

A. Experimental Setup

The experimental setup consists of two physically isolated Linux machines, one acting as the client and the other as the server, connected via a dedicated Gigabit Ethernet link. The architecture is shown in Fig. 1. The client machine is responsible for workload generation, packet capture, and post-processing, while the server hosts minimal echo services for REST and gRPC.

Both systems are configured with identical hardware and software to eliminate performance variability due to system heterogeneity, as summarized in Table II.

The machines are directly connected using static IP addresses (10.10.10.1/30 and 10.10.10.2/30), ensuring a controlled and contention-free network environment.

The client executes k6 for workload generation, tcpdump and tshark for packet capture and analysis, and Python scripts for aggregation and plotting. The server hosts a REST echo service and a gRPC echo service.

All services are implemented in Go, chosen for its low runtime overhead and efficient execution model to ensure that measurements reflect protocol-level behavior only. The use of two dedicated machines, a direct Ethernet link, and minimal application logic ensures that the observed overheads arise solely from encoding, framing, and serialization rather than external system noise.

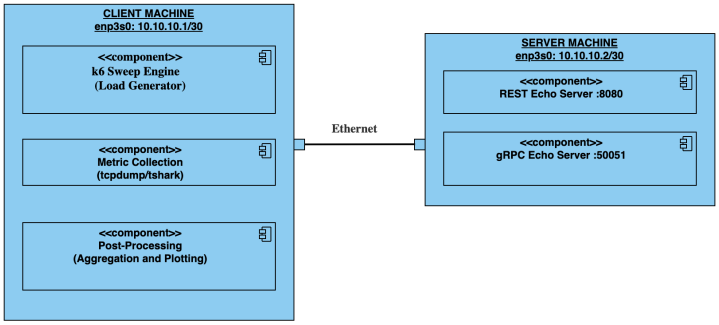

Fig. 1. Experimental setup comprising isolated client and server machines connected via a dedicated Ethernet link.

B. Input Configuration

The experiment explores a structured input space defined as

$$\mathcal{I} = P \times S \times R, \tag{1}$$

where P denotes the set of payload sizes, S denotes the set of payload structures, and R denotes the set of protocols.

In this study, these parameters are instantiated as follows:

- Payload sizes:

$$P = \{32, 64, 128, 512, 1024, 8192\} \text{ bytes}, \tag{2}$$

  with $|P| = 6$.
- Payload structures:

$$S = \{\text{Flat}, \text{Nested}, \text{Wide}, \text{Array}\}, \tag{3}$$

  with $|S| = 4$.
- Protocols:

$$R = \{\text{REST}, \text{gRPC}\}, \tag{4}$$

  with $|R| = 2$.

The total configuration space is therefore

$$|\mathcal{I}| = |P| \times |S| \times |R| = 6 \times 4 \times 2 = 48. \tag{5}$$

Thus, a total of 48 unique configurations are evaluated.

For time experiments, each configuration runs for R = 1000 requests in a single k6 run. The first 10% of samples are discarded as warm-up, and the mean is reported per configuration.

Concurrency is fixed at c = 1 to isolate serialization cost from queueing and scheduling effects.

For space experiments, each configuration is executed for 100 requests over a single connection to amortize connection setup overhead and obtain a stable estimate of per-request wire cost.

To evaluate sensitivity to data shape, four representative payload structures are used, as summarized in Table III.

These structures capture different encoding characteristics, particularly the verbosity of JSON and the field encoding efficiency of Protobuf.

TABLE III
PAYLOAD STRUCTURE CATEGORIES

Structure	Description
Flat	Single-level key-value pairs
Nested	Multi-level hierarchical objects
Wide	Large number of fields at a single level
Array	Repeated homogeneous elements

TABLE IV
SPACE OVERHEAD METRICS

Metric	Source	Description
Logical size	Client (`k6`)	Raw in-memory payload
Wire bytes	`tshark`	Total transmitted bytes
Body bytes	`tshark`	Encoded payload
Header bytes	Derived	Protocol overhead

C. Space Overhead Measurement

Space overhead is decomposed into two components:

- **Encoding overhead**: difference introduced by payload representation (JSON vs. Protobuf).
- **Framing overhead**: additional bytes introduced by transport protocols (HTTP/1.1 vs. HTTP/2).

Measurement Pipeline Space metrics are collected using a three-stage pipeline.

1) Traffic Generation: The client generates requests using a `k6` script (`sweep.js`), executing 100 iterations per configuration. Logical payload size is computed prior to encoding, and for REST, JSON-encoded size is also recorded.

2) Packet Capture: Network traffic is captured using `tcpdump` during execution. Traffic is filtered by server IP and ports 8080 and 50051, and the captured data are stored as `.pcap` files. This ensures full visibility into HTTP/1.1 headers, HTTP/2 frames, and gRPC framing.

3) Post-Capture Analysis: Captured traces are processed using `tshark` to extract wire bytes (`tcp.len`), total TCP payload comprising headers, body, and framing; body bytes, using `http.content_length` for REST and `grpc.message_length` for gRPC; and header bytes, computed as

$$\text{Header Bytes} = \text{Wire Bytes} - \text{Body Bytes}. \tag{6}$$

The collected space metrics are summarized in Table IV.

All measurements are averaged over 100 iterations to reduce variance.

D. Time Overhead Measurement

Time overhead captures the serialization and deserialization cost introduced by the protocols.

Measurement Approach Serialization and deserialization are measured using explicit in-code instrumentation within the Go server and high-resolution timers (`time.Now()` and `time.Since()`). For REST, deserialization is measured using `json.Unmarshal` and serialization using `json.Marshal`. For gRPC, serialization and deserialization are normally handled internally by the framework; to ensure comparability, equivalent operations (`proto.Unmarshal` and `proto.Marshal`) are explicitly executed within the handler and timed.

The total server-side time is defined as

$$T_{\text{server}} = T_{\text{deser}} + T_{\text{ser}}. \tag{7}$$

Measurement Scope Serialization and deserialization are measured only on the server side. This design choice is motivated by two considerations. First, for fairness, the client runs in JavaScript (`k6`) while the server runs in Go; measuring both sides would introduce language bias. Second, client-side gRPC encoding occurs within internal runtime layers and cannot be measured precisely. Thus, server-side timing provides a consistent and comparable measurement baseline, while client-side serialization and deserialization are performed but not timed.

E. Output Metrics

The methodology produces four primary output metrics that are plotted and analysed in Section III.

Space Metrics

1) **Overhead ratio versus payload size**

$$O_1 = \frac{\text{Wire Bytes}}{\text{Logical Size}}, \tag{8}$$

where *Wire Bytes* denotes the total transmitted bytes observed on the wire and *Logical Size* denotes the raw application payload before protocol-specific encoding.

2) **Encoding overhead**

$$O_2 = \frac{\text{Body Bytes}}{\text{Logical Size}}, \tag{9}$$

where *Body Bytes* denotes the serialized payload size after JSON or Protobuf encoding.

3) **Framing overhead**

$$O_3 = \text{Wire Bytes} - \text{Body Bytes}, \tag{10}$$

which captures the bytes introduced by transport headers and protocol framing.

Time Metric

4) **Serialization/deserialization overhead**

$$T_{\text{server}} = T_{\text{ser}} + T_{\text{deser}}, \tag{11}$$

where T_{ser} is server-side serialization time and T_{deser} is server-side deserialization time.

This methodology enables a controlled experimentation for systematic decomposition of communication overhead into space, comprising encoding and framing, and time, comprising serialization and deserialization and provides insight into protocol overheads.

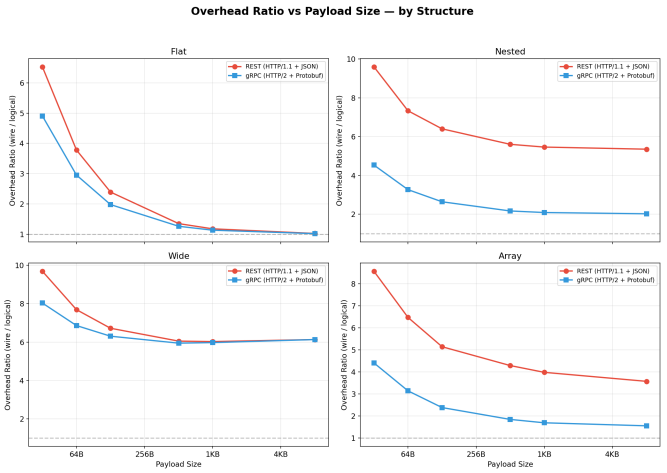

Fig. 2. Overhead ratio across payload sizes and payload structures for REST and gRPC.

III. Results and Analysis

A. Overhead Ratio

Observations: The overhead ratio decreases consistently as payload size increases across all structures. For small payloads, both protocols exhibit substantial overhead, which diminishes progressively with larger payloads.

gRPC consistently demonstrates lower overhead than REST across all configurations. The difference between the two protocols is more noticeable at smaller payload sizes and gradually reduces as payload size increases, though it remains observable.

Payload structure significantly influences the asymptotic behavior of the overhead ratio. Flat structures approach near-optimal efficiency as payload size increases, while Nested, Wide, and Array structures retain noticeable overhead even at larger payload sizes. Among these, Wide structures exhibit the most persistent overhead, showing limited reduction with increasing payload size.

Explanation: At small payload sizes, fixed protocol overhead dominates, resulting in high ratios. As payload size increases, this fixed cost becomes amortized, and the ratio increasingly reflects encoding efficiency.

Differences across structures arise from how metadata scales with payload representation. Flat structures involve minimal metadata and therefore achieve near-optimal efficiency. In contrast, Nested and Wide structures introduce repeated structural elements, which sustain higher overhead even at larger payload sizes.

The lower overhead observed for gRPC is attributable to its use of compact binary encoding and more efficient transport framing.

B. Encoding Overhead

Observations: Encoding overhead is strongly influenced by payload structure. Flat structures exhibit minimal overhead and converge toward optimal encoding efficiency as payload size increases. In contrast, Nested and Wide structures retain significant overhead across all payload sizes. Array structures show intermediate behavior, with overhead decreasing as payload size increases but remaining higher than flat structures.

The relative advantage of gRPC varies depending on structure. It is most noticeable for Nested and Array structures, while for Wide structures, the difference between protocols is minimal. Thus, for certain structures, encoding overhead remains relatively stable across payload sizes, while for others it decreases or slightly increases.

Explanation: This metric isolates the cost introduced by serialization. It includes structural metadata such as field names, delimiters, and schema identifiers.

Flat structures incur minimal overhead due to limited metadata. Nested structures amplify overhead in JSON due to repeated field names and hierarchical representation, whereas Protobuf reduces this cost through compact encoding.

Wide structures involve a large number of fields, causing metadata to scale with payload size. This prevents effective amortization and leads to persistently high overhead across both protocols.

Array structures benefit from repeated schema usage, allowing more efficient representation in Protobuf compared with JSON's textual format.

C. Framing Overhead

Observations: Framing overhead remains largely constant across all payload sizes and structures. The amount of header data transmitted per request does not vary meaningfully with payload size or data structure.

gRPC consistently incurs lower framing overhead than REST. This difference remains stable across all configurations. Minor variations are observed at larger payload sizes, particularly for gRPC, where framing overhead shows slight increases.

Explanation: This overhead is inherently fixed and independent of payload size or structure. It includes HTTP headers in REST and binary frame metadata in gRPC.

REST incurs higher overhead due to verbose textual headers, while gRPC benefits from binary framing and header compression mechanisms. The slight increase observed at larger payload sizes is attributable to segmentation effects, where larger payloads are split across multiple frames, introducing additional framing metadata.

Framing overhead dominates total overhead at small payload sizes but becomes negligible as payload size increases.

D. Serialization/Deserialization Overhead

Observations: Serialization and deserialization time increases with payload size across all structures. The increase is gradual at smaller payload sizes and becomes more pronounced as payload size grows.

gRPC consistently demonstrates lower computational overhead than REST. The performance difference becomes more significant at larger payload sizes.

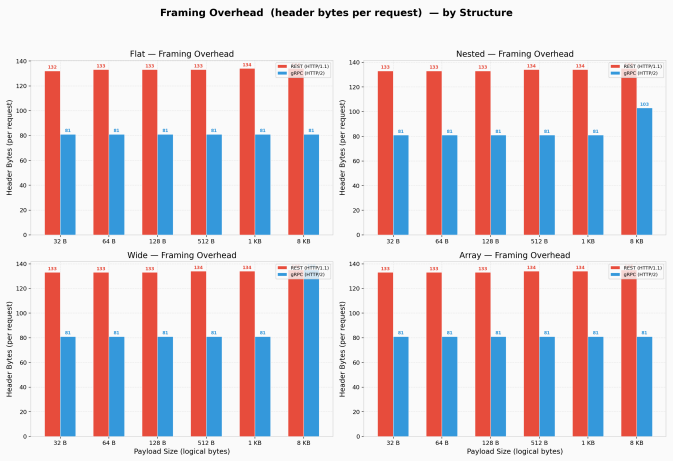

Fig. 3. Framing overhead across payload sizes and payload structures for REST and gRPC.

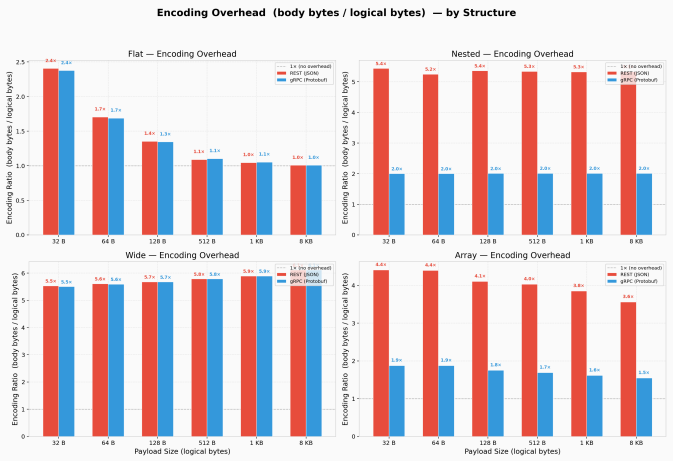

Fig. 4. Encoding overhead across payload sizes and payload structures for REST and gRPC.

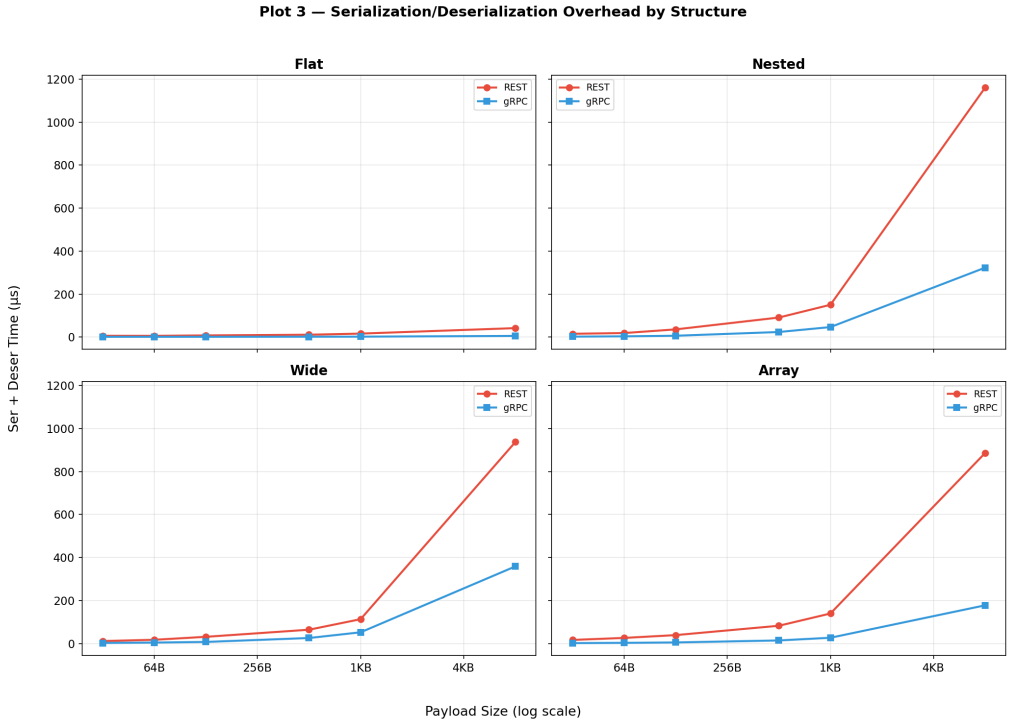

Fig. 5. Serialization and deserialization overhead across payload sizes and payload structures for REST and gRPC.

Payload structure has a strong influence on computational cost. Flat structures exhibit minimal overhead and scale efficiently with payload size. In contrast, Nested and Wide structures incur significantly higher overhead, with Nested structures showing the highest cost. Array structures exhibit moderate overhead that increases with payload size.

Explanation: JSON-based REST incurs higher overhead due to string parsing, tokenization, and dynamic memory allocation. These operations introduce higher constant factors and are sensitive to structural complexity.

Protobuf uses a binary, schema-driven encoding that enables more efficient parsing and reduces computational overhead. This results in more predictable and scalable performance.

Structural complexity directly impacts processing cost. Nested structures require recursive traversal and repeated allocations, leading to higher overhead. Wide structures increase the number of fields processed, while array structures involve repeated element handling.

The more noticeable increase at larger payload sizes is likely due to memory allocation overhead and reduced cache efficiency, which disproportionately affect JSON processing.

Summary: The results demonstrate the usefulness of the proposed decomposition:

- Framing overhead is constant and dominates at small payload sizes.
- Encoding overhead determines asymptotic space efficiency and depends on payload structure.
- Serialization/deserialization overhead captures computational cost and scales with both size and structural complexity.

This decomposition enables a clear separation of protocol overhead into interpretable components, providing insight beyond end-to-end measurements like latency, CPU Utilization, memory consumption.

IV. Conclusion

This work presented a systematic decomposition of communication overhead in REST and gRPC into distinct space (encoding and framing) and time (serialization/deserialization) components. By using a controlled, two-machine setup with minimal application logic, the study isolates protocol-level behavior which enables precise attribution of overhead sources.

From a space perspective, framing overhead is constant and independent of payload characteristics, while encoding overhead dominates at larger payload sizes and is strongly influenced by payload structure.

From a time perspective, serialization and deserialization cost increases with both payload size and structural complexity, with JSON incurring higher computational overhead due to parsing and allocation, and Protobuf providing more efficient and scalable performance. Overall, gRPC demonstrates better efficiency in both space and time, particularly for complex and large payloads, while differences are less pronounced for simple structures.

These observations have practical implications for system design. For small payloads, reducing framing overhead is critical, whereas for larger or more complex data, encoding efficiency and serialization cost become dominant factors. The results also highlight that payload structure plays a crucial role in determining performance, making data modeling decisions as important as protocol selection in distributed systems.

The study is limited to a controlled environment and does not capture real-world factors such as concurrency, network variability, or system-level interactions. Future work can extend this analysis by exploring a broader configuration space and correlating the decomposed overhead components with end-to-end metrics such as latency, throughput, and CPU utilization in more realistic deployment settings.

Acknowledgment

The authors thank the COMET Lab, IIIT Bangalore for providing the necessary infrastructure and equipment that enabled this experimental study.

References

[1] R. Sannapureddy, “Cloud-native enterprise integration: Architectures, challenges, and best practices,” Journal of Engineering and Computer Sciences, vol. 4, no. 8, pp. 775–781, 2025.

[2] L. D. S. B. Weerasinghe and I. Perera, “Evaluating the inter-service communication on microservice architecture,” in Proc. 7th Int. Conf. Information Technology Research (ICITR), 2022.

[3] V. Muniyandi, “Utilizing gRPC for high-performance inter-service communication in .NET,” Available at SSRN 5381441, 2025.

[4] B. Ciftci and B. Ciloglugil, “A review of comparative studies on performance evaluation of communication mechanisms for microservices, in extitProc. Int. Conf. Computational Science and Its Applications. Cham, Switzerland: Springer Nature Switzerland, 2025.

[5] I. G. Buzhin *et al.*, “Comparative analysis of REST and gRPC used in the monitoring system of communication network virtualized infrastructure,” *T-Comm—Telecommunications and Transport*, vol. 17, no. 4, pp. 50–55, 2023.

[6] Niswar, Muhammad, et al. ”Performance evaluation of microservices communication with REST, GraphQL, and gRPC.” International Journal of Electronics and Telecommunication 70.2 (2024): 429-436.

[7] Arora, Shruti, et al. ”A Comparative Analysis of Communication Efficiency: REST vs. gRPC in Microservice-Based Ecosystems.” 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP). IEEE, 2024.

[8] Díaz, Roberto Albeiro Pava, Dario Alejandro Segura Torres, and Bryan Steven Perez Salas. ”Performance-Based Selection Methodology for Web Communication Protocols.” 2025 9th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT). IEEE, 2025.

[9] Zaniewski, Patryk, and Rados law Roszczyk. ”Comparative review of selected Internet communication protocols.” (2024)

[10] Muniyandi, Venkatesh. ”Utilizing Grpc For High-Performance Inter-Service Communication In. NET.” Available at SSRN 5381441 (2025).

[11] Chamas, Carolina Luiza, Daniel Cordeiro, and Marcelo Medeiros Eler. ”Comparing REST, SOAP, Socket and gRPC in computation offloading of mobile applications: An energy cost analysis.” 2017 IEEE 9th Latin-American Conference on Communications (LATINCOM). IEEE, 2017.

[12] Kumar, Prajwal Kiran, et al. ”Performance characterization of communication protocols in microservice applications.” 2021 International Conference on Smart Applications, Communications and Networking (SmartNets). IEEE, 2021.